YG Health & Social Services Backend — Code Quality & Security Audit

Prepared for: Project stakeholder (incoming maintainer / customer decision-maker) **Prepared by:** Michael Johnson **Date:** 2026-06-12 **Repository:** `hss-backend` (API: Node.js / Express / TypeScript / Oracle; Web: Vue 2) **Status:** Inherited codebase — pre-engagement assessment

---

1. Purpose of this document

This document is an independent assessment of the inherited `hss-backend` codebase. It catalogues defects in security, correctness, and maintainability so that an informed decision can be made about **which items to fund for remediation before further development proceeds**.

Each item includes a plain-language explanation of **why it is a problem** and its **realistic impact**, so that non-engineering stakeholders can weigh the risk. Issues are grouped by severity. The application handles **personal health information (PHI) and personally identifiable information (PII)** for a government health-services program, so confidentiality and integrity defects carry elevated regulatory and reputational consequences.

> **Bottom line:** The application in its current state should be treated as **not production-ready from a security standpoint.** The most serious issue — covered in CRIT-01 — is that the great majority of the API, including endpoints that return health records and uploaded documents, can be accessed **without any login**. This single class of defect would be expected to fail any penetration test or privacy review outright.

---

2. Severity definitions

Severity	Meaning
Critical	Directly exploitable; leads to exposure/loss of PHI/PII, full data compromise, or remote code execution. Fix before any production use.
High	Serious weakness that is exploitable under common conditions or significantly amplifies other issues. Fix in the current remediation cycle.
Medium	Real defect with limited exploitability or moderate impact; degrades security posture, reliability, or maintainability. Schedule for near-term fix.
Low	Hygiene, maintainability, or defense-in-depth gap. Low individual risk; worth correcting opportunistically.

---

3. Executive summary

Severity	Count
Critical	7
High	9
Medium	10
Low	9
Total	**35**

Headline themes:

1. **Broken authentication & access control (Critical).** Authentication is configured as optional (`authRequired: false`) and is not enforced on the routers. Nearly every endpoint across the dental, constellation, HIPMA, midwifery, and general modules is reachable by an **anonymous, unauthenticated caller** — including reading, exporting, downloading files for, modifying, and closing health-services records.
2. **SQL injection (Critical).** User-supplied file data is concatenated directly into an Oracle PL/SQL block in the HIPMA submission endpoint and in two migration scripts, allowing arbitrary database commands.
3. **Insecure deployment (Critical/High).** Real secrets (`.env`) are baked into the Docker image, containers run as root, and a 112 MB Oracle binary is committed to source control.
4. **Vulnerable dependencies (High).** Multiple packages with published CVEs (notably `axios 0.24.0`, `xlsx 0.18.5`) and end-of-life frameworks (Vue 2, Node 13/16).
5. **Systemic reliability defects (High/Medium).** Async/await misuse causing double HTTP responses and lost errors, ineffective rate limiting, no automated test coverage, and information-disclosing

error handlers.

A penetration test against this application would be expected to flag items across **OWASP Top 10 A01 (Broken Access Control), A02 (Cryptographic/Secrets), A03 (Injection), A05 (Security Misconfiguration), A06 (Vulnerable Components), and A09 (Logging/Monitoring)**.

---

4. Critical findings

CRIT-01 — Authentication is not enforced; most of the API is anonymously accessible

Location: `src/api/routes/auth.ts:53-54` (`authRequired: false`); `src/api/index.ts:56-58` (routers mounted with no auth gate); all routes in `routes/hipma.ts` , `routes/midwifery.ts` , `routes/general.ts` , and all but one route each in `routes/dental.ts` and `routes/constellation.ts` .

Why it's a problem: The OpenID Connect middleware is configured with `authRequired: false` . The only global middleware (`auth.ts:68-86`) *populates* the user object when a session happens to exist but **never rejects** anonymous requests. A route is therefore protected only if it explicitly lists the `EnsureAuthenticated` handler — and almost none do. The HIPMA, midwifery, and general modules import `EnsureAuthenticated` but never use it. Only `user.ts` and a single `/show/:id` route per module are gated.

Impact: Anyone who can reach the server can list, read, export, download attachments for, modify, and close submissions containing names, dates of birth, health-card numbers, diagnoses, and addresses. This is a mass PHI/PII breach exposure and a privacy-law violation. **This is the single most important item in this document.**

Representative unprotected endpoints: `GET /api/hipma/show/:id` , `GET /api/hipma/downloadFile/:id` , `POST /api/dental/export` , `PATCH /api/dental/changeStatus` , `GET /api/constellation/duplicates/details/:id` , `GET /api/general/audit/data/:event_type` , and dozens more.

Recommendation: Enforce authentication at the router-mount level in `index.ts` (apply `EnsureAuthenticated` to every `/api/*` router except genuinely public citizen-submission endpoints), then layer `checkPermissions( ... )` per route. Treat public submission endpoints explicitly and minimally.

CRIT-02 — SQL injection via file data concatenated into a PL/SQL block

Location: `src/api/routes/hipma.ts:504-520` (and the equivalent helper); also `src/api/dataMigration.js:851` and `src/api/dentalMigrationByIDs.js:334` .

Why it's a problem: User-supplied base64 file content (`value.file_data`) is split into 4000-character chunks and each chunk is concatenated **directly into a single-quoted Oracle literal**: `query = query +`

`" DBMS_LOB.APPEND(v_long_text, to_blob(utl_raw.cast_to_raw('" + element + "'))); "` which is then executed via `db.raw( ... )`. A single quote (or a crafted `'); <statement>; --`) in the payload breaks out of the literal and injects arbitrary PL/SQL that runs with the application's database privileges. The surrounding `INSERT` uses safe `?` bindings, but this fragment does not.

Impact: Full database compromise — read, modify, or destroy PHI; potentially escalate within the database. Because the endpoint is also unauthenticated (CRIT-01) and returns raw error text (HIGH-02), it is an easily weaponized, error-based injection.

Recommendation: Never build SQL by string concatenation of user data. Bind the file bytes as parameters, or use the driver's native LOB/streaming API (`oracledb` BLOB binding) to write attachment data.

CRIT-03 — Insecure Direct Object References (IDOR) on record and file endpoints

Location: e.g. `dental.ts:845` & `constellation.ts:865` (`/duplicates/details/:id`), `dental.ts:1121` (`/downloadFile/:id`), `hipma.ts:603` (`/downloadFile/:id`), `hipma.ts:221` / `midwifery.ts:259` (`/show/:id`).

Why it's a problem: Records and uploaded files are fetched purely by an integer ID taken from the request, with no check that the caller is permitted to see that specific record. IDs are sequential and guessable.

Impact: Even if authentication were added, any user could enumerate IDs and read every other applicant's records and documents (horizontal privilege escalation). Combined with CRIT-01, it is fully anonymous today.

Recommendation: Add per-record authorization checks (ownership or role/permission scoping) on every endpoint that accepts a record identifier.

CRIT-04 — Unauthenticated mass-assignment update of arbitrary records

Location: `src/api/routes/dental.ts:1585` (`PATCH /update`) → `dental.ts:1915` (`db( ... ).update(data).where("ID", idSubmission)`); dependents update at `dental.ts:1832-1834`; `storeComments` user-spoofing at `dental.ts:1541-1553`.

Why it's a problem: The request body (`req.body.params.data`) is passed wholesale into a knex `update()`, and the target row ID is also attacker-controlled, with no authentication, no validation, and no allow-list of writable columns. `storeComments` takes the acting `USER_ID` from the body and writes it into the record **and the audit log**.

Impact: An anonymous attacker can overwrite any column of any dental submission and forge audit-trail entries (repudiation). This undermines both data integrity and the reliability of the audit log.

Recommendation: Authenticate and authorize; validate input; explicitly allow-list updatable fields; derive the acting user from the authenticated session, never from the request body.

CRIT-05 — Path traversal / arbitrary file write to the served web root

Location: `src/api/routes/hipma.ts:603-619` (`fs.writeFileSync(pathFile, buffer)` with unsanitized `file_type` / `file_name`); `dental.ts:1121-1143` (writes BLOB into the statically served `dist/web` directory).

Why it's a problem: Download endpoints reconstruct a filesystem path from client-influenced values (`file_name` , `file_type`) that were stored unsanitized at submission time, then write bytes to disk inside the directory that Express serves statically (`index.ts:75`). A crafted `file_type` / `file_name` containing `../` can write attacker-controlled content outside the intended folder — e.g. overwriting served JavaScript.

Impact: Arbitrary file write into the web root can lead to front-end defacement or, in the worst case, remote code execution / stored XSS served to staff browsers. Unauthenticated (CRIT-01).

Recommendation: Sanitize and validate file names/types on upload; store attachments outside the web root; stream downloads from the database to the response rather than writing to a served directory; never derive disk paths from client input.

CRIT-06 — `SKIP_PERMISSIONS` flag silently disables authorization when set to the string `"false"`

Location: `src/api/config.ts:30` (`export const SKIP_PERMISSIONS = process.env.SKIP_PERMISSIONS || false;`) and `src/api/middleware/permissions.ts:11-15` .

Why it's a problem: Environment variables are strings. The expression `process.env.SKIP_PERMISSIONS || false` evaluates to the **non-empty string** `"false"` when an operator sets `SKIP_PERMISSIONS=false` to *disable* the bypass — and a non-empty string is truthy. The intent (turn the bypass off) produces the opposite effect (turn the bypass on), disabling all permission checks. Additionally, in `checkPermissions` the bypass branch calls `next()` **without** `return` , so execution continues and the handler can both proceed and later send a `401` .

Impact: A well-meaning configuration setting can silently remove authorization across the application. This is a latent footgun that is very likely to be triggered in practice.

Recommendation: Parse booleans explicitly (`=== 'true'`); default to the secure value; `return next()` from the bypass branch; ideally remove the bypass entirely outside local development.

CRIT-07 — Production secrets baked into the Docker image

Location: root `Dockerfile:28` (`COPY src/api/.env* ./`); `src/api/docker-compose.yml:11` (mounts `./.env`).

Why it's a problem: The real `.env` (database passwords, OIDC `CLIENT_SECRET` , session/Redis secrets) is copied into an image layer. Anyone who can pull the published image (`ghcr.io/...`) can extract the secrets via `docker history` / layer inspection.

Impact: Disclosure of all backend credentials to anyone with image access — a direct path to database and identity-provider compromise.

Recommendation: Never copy `.env` into images. Inject secrets at runtime via the orchestrator's secret store / environment, and add `.env` to `.dockerignore` .

---

5. High findings

HIGH-01 — `RequiresRoleAdmin` admin check is broken (fails open)

Location: `src/api/middleware/index.ts:24-30` .

Why it's a problem: The guard reads `if (req.user && req.user.roles.indexOf('Admin') == -1)` . It checks for the role string `'Admin'` , but roles are defined elsewhere as `'Administrator'` (`middleware/authorization.ts:6`). Worse, when `req.user` is **undefined** (an unauthenticated request) the `if` is skipped and `next()` runs — the request passes through. The middleware therefore both rejects legitimate admins and admits anonymous users.

Impact: Any admin-gated route using this guard is not actually protected. Fails open.

Recommendation: Use the canonical role constant; treat a missing user as unauthorized; add a unit test.

HIGH-02 — Information disclosure: raw exceptions returned to the client

Location: `hipma.ts:538` , `midwifery.ts:544` , `constellation.ts:510` , `dental.ts:1434` , and similar (`message: 'Request could not be processed ' + e`).

Why it's a problem: Caught exceptions — including Oracle error text that echoes SQL fragments, table/column names, and bind values — are concatenated into the HTTP response body.

Impact: Leaks internal schema and query structure to attackers and turns the injectable endpoint (CRIT-02) into an error-based injection oracle.

Recommendation: Return a generic error message and correlation ID to clients; log details server-side only.

HIGH-03 — Rate limiting is ineffective and misordered

Location: `src/api/index.ts:62-69` (limiter `max: 5000`/min, and `app.use(limiter)` registered **after** routes are mounted at lines 56-58); duplicate limiters in `auth.ts:39-42` and per-router (`user.ts:10-13`), all `max: 5000`.

Why it's a problem: Express runs middleware in registration order, so a limiter mounted after the routers does not protect those routers. Regardless, a ceiling of 5,000 requests/minute per client is effectively no limit. Multiple redundant limiters add confusion without protection.

Impact: No meaningful protection against brute force, scraping, or denial-of-service — particularly relevant given the unauthenticated endpoints (CRIT-01).

Recommendation: Mount a single limiter **before** routes with a realistic threshold; apply stricter limits to auth and export endpoints.

HIGH-04 — Vulnerable and end-of-life dependencies (see §8 for the full list)

Location: `src/api/package.json`, `src/web/package.json`.

Why it's a problem: `axios 0.24.0` (SSRF / credential-leak / ReDoS CVEs), `xlsx 0.18.5` (prototype pollution CVE-2023-30533, ReDoS CVE-2024-22363 — no fixed npm release), `mysql 2.18.1` (unmaintained), and the front-end on **Vue 2 / Vuetify 2 (end-of-life, no security patches)**.

Impact: Known, publicly documented vulnerabilities present in the deployed stack; an `npm audit` will flag several High/Critical advisories.

Recommendation: Upgrade `axios` to a current 1.x; migrate `xlsx` to the vendor's maintained distribution; replace `mysql` with `mysql2`; plan a Vue 3 / Vuetify 3 migration.

HIGH-05 — Async `forEach` / `_.forEach` misuse causing double responses and lost errors

Location: `hipma.ts:495-532`, `constellation.ts:822-844` & `483-504`, `dental.ts:1364-1377`, and similar.

Why it's a problem: `_.forEach(..., async () => { await db... })` does not await its callbacks, so database writes run fire-and-forget; a shadowed inner `var filesSaved` is never observed by the outer success check; and both the loop's error branch and the success branch can call `res.json(...)`, producing "headers already sent" crashes and false-success responses.

Impact: Intermittent crashes, file writes reported as successful when they failed, and unhandled promise rejections. Unreliable under real load.

Recommendation: Use `for ... of` with `await` (or `Promise.all` over `.map`); ensure exactly one response per request; `return` after sending.

HIGH-06 — Containers run as root

Location: root `Dockerfile` and `src/api/Dockerfile` — no `USER` directive.

Why it's a problem: The application process runs as UID 0 inside the container. A code-execution or container-escape bug then operates with root in the namespace.

Impact: Amplifies the blast radius of any other vulnerability.

Recommendation: Create and switch to a non-root user; set least-privilege filesystem ownership.

HIGH-07 — 112 MB Oracle Instant Client binaries committed to source control

Location: `instantclient_21_9/` (≈37 tracked files; re-added in `Dockerfile:10`).

Why it's a problem: Large proprietary `.so` binaries are stored in git, bloating every clone, risking shipment of an unpatched/licensed binary, and duplicating what the Dockerfile already downloads at build time.

Impact: Repository bloat, licensing exposure, and stale/unpatched native libraries.

Recommendation: Remove from git history, add to `.gitignore` / `.dockerignore`, and fetch at build time.

HIGH-08 — Excessive request body limit enables denial of service

Location: `src/api/index.ts:20-22` (`limit: '50mb'` on JSON and urlencoded bodies).

Why it's a problem: A 50 MB body cap on unauthenticated endpoints lets a small number of clients exhaust memory/CPU (JSON parsing, base64 decoding, BLOB handling).

Impact: Cheap denial-of-service; compounded by the ineffective rate limiting (HIGH-03).

Recommendation: Lower the global limit to a realistic value; raise it only on the specific endpoints that need large uploads, behind authentication.

HIGH-09 — Helmet security headers largely disabled

Location: `src/api/index.ts:24` (`//app.use(helmet());` commented out; only a CSP applied at 25-40).

Why it's a problem: The full Helmet middleware is disabled, so default protections (HSTS, `X-Content-Type-Options`, frameguard, etc.) are absent; only a Content-Security-Policy remains.

Impact: Weakened browser-side defenses (clickjacking, MIME sniffing, transport downgrade).

Recommendation: Enable `helmet()` and configure the CSP through it.

6. Medium findings

MED-01 — No automated test coverage

Location: `src/api` — no `*.test.ts` / `*.spec.ts` exist, though `jest`, `ts-jest`, and `supertest` are configured (`package.json:6`, `jest.config.js`).

Why it's a problem: There is no regression safety net; refactoring (including the security fixes above) cannot be validated automatically.

Recommendation: Add tests for auth/authorization, input validation, and the data layer as part of remediation.

MED-02 — End-of-life base images and runtimes

Location: root `Dockerfile:1,14` (`oraclelinux:7-slim`, Node 16); `src/api/Dockerfile:1,14` (`oraclelinux:7-slim`, Node **13**).

Why it's a problem: Oracle Linux 7, Node 13, and Node 16 are end-of-life and receive no security patches; the two Dockerfiles even disagree on the Node version.

Recommendation: Standardize on a supported LTS Node and a maintained base image.

MED-03 — Redis published to the host without authentication

Location: `docker-compose.yml` / `docker-compose.production.yml:11-12` (`6379:6379`); custom Redis image sets no `requirepass`. Also a variable collision: `${HOST_PORT:-6379}` reuses the same variable as the web port mapping.

Why it's a problem: Exposing an unauthenticated Redis on the host risks data access / abuse if the host firewall is misconfigured; the session store lives in Redis.

Recommendation: Do not publish Redis to the host; require a password; bind to the internal network only.

MED-04 — Race condition on a shared module-level DB client

Location: all four route files reassign a single module-level `let db = await helper.getOracleClient(db, ...)` on nearly every request (e.g. `hipma.ts:94,190,224`; `midwifery.ts:94,420,453`).

Why it's a problem: Concurrent requests mutate shared connection state, which can cause intermittent wrong-connection use or query errors under load.

Recommendation: Obtain a connection per request (local variable) rather than mutating module state.

MED-05 — Sort parameters not allow-listed (inconsistent across modules)

Location: `dental.ts:155-156` and `hipma.ts:137` use `sortBy` / `sortOrder` from the request with no allow-list, whereas `constellation.ts` and `midwifery.ts` do allow-list them.

Why it's a problem: Although knex quotes identifiers, unvalidated sort fields allow column probing and error-based reconnaissance, and the inconsistency signals uneven review.

Recommendation: Allow-list sortable columns and directions everywhere.

MED-06 — File upload validation is weak / not enforced

Location: `hipma.ts:1190-1237` (`saveFile`), `dental.ts:2064-2103` .

Why it's a problem: The extension allow-list is bypassed by falling back to the client-supplied filename extension; the size check happens **after** the full buffer is written to disk; file metadata is stored unsanitized (feeds CRIT-05).

Recommendation: Enforce a strict server-side type/size allow-list before persisting; reject (don't fall back) on disallowed types.

MED-07 — Session cookie hardening and secret reuse

Location: `src/api/routes/auth.ts:44-50` .

Why it's a problem: The `express-session` configuration sets no explicit cookie flags (`secure` , `httpOnly` , `sameSite` , `maxAge`), and the session `secret` reuses the Redis password (`REDIS_CONFIG.secret`) rather than a dedicated session secret.

Impact: Session cookies may be transmittable over plain HTTP and accessible to scripts depending on defaults; secret reuse couples two trust domains.

Recommendation: Set `cookie: { secure: true, httpOnly: true, sameSite: 'lax', maxAge }` ; use a dedicated, high-entropy session secret.

MED-08 — Declared input validators are never evaluated

Location: routes attach `param( ... ).isInt()` etc. (e.g. `hipma.ts:183,221`) but no handler calls `validationResult(req)` / the `ReturnValidationErrors` middleware.

Why it's a problem: Validation is decorative — invalid input passes through (mitigated only incidentally by `Number()` coercion). POST/PUT bodies have no validation at all.

Recommendation: Wire `ReturnValidationErrors` into the routes and add body validators.

MED-09 — CI build-cache poisoning from fork pull requests

Location: `.github/workflows/docker-publish.yml:9-16,93` (builds on `pull_request`, `cache-to: type=gha,mode=max`).

Why it's a problem: Untrusted PRs build attacker-controlled Dockerfiles and can poison the shared GitHub Actions cache consumed by trusted `main` builds. (Push/login are correctly gated, limiting direct registry writes.)

Recommendation: Disable cache export for fork PRs, or scope caches by trust level.

MED-10 — Deprecated CI action versions

Location: `.github/workflows/codeql-analysis.yml:38,42,53,67` (`actions/checkout@v2`, `github/codeql-action/*@v1`).

Why it's a problem: These versions run on a retired Node 12 runner and the retired CodeQL v1; scans may silently degrade or fail, leaving the team with false assurance.

Recommendation: Upgrade to current action versions.

---

7. Low findings

LOW-01 — Pervasive `console.log(e)` may write PHI/PII to logs

`hipma.ts:46`, `midwifery.ts:46`, `dental.ts:51`, `auth.ts:80`, and many others (often tagged `// debug if needed`). Exceptions over health-data payloads are logged verbatim. Use structured logging that redacts sensitive fields.

LOW-02 — Weak randomness for identifiers and filenames

`Math.random()`-based "safe" filenames (`hipma.ts:612,1203`) and `uniqid()` confirmation numbers (`hipma.ts:1176`, `midwifery.ts:1262`) are predictable. Use `crypto.randomUUID()` / `crypto.randomBytes`.

LOW-03 — Weak default credentials in committed `.env.sample`

`src/api/.env.sample:13,22,30,33-34` ship `DB_PASS=bizont`, `SECRET=Along, ...`, `root/root`. These are samples (the real `.env` is correctly gitignored), but copy-paste defaults invite weak production secrets.

LOW-04 — Broken/malformed deployment entries

`src/api/docker-compose.yml:9` has a stray `]` in a volume entry; `src/api/Dockerfile:31` uses `CMD ["npm run start"]` (single exec-form arg); `package.json:11` `"start": "node dist/index.ts"` points at a `.ts` file rather than the compiled `.js`.

LOW-05 — `.dockerignore` omits `.env`, `.git`, and `instantclient_21_9`

The build context needlessly includes secrets and the 112 MB binary tree (reinforces CRIT-07 and HIGH-07).

LOW-06 — Dead / misleading "truncation" safeguards

`hipma.ts:703-717`, `general.ts:389-403` compute a `bufferQuery` that is never used; the "5MB/1MB limit" comments are misleading and only affect an audit-log buffer.

LOW-07 — Data-integrity bug building a JSON string by concatenation

`constellation.ts:1321-1327` builds JSON via `"["+modelValues+',"'+others+'"]'` without escaping; a value containing `"` corrupts the stored diagnosis blob.

LOW-08 — Commented-out / dead security code

`index.ts:24` (helmet), `auth.ts:63` (logout route), `index.ts:71-73`. Remove dead code or restore intended protections.

LOW-09 — Validation/success returned without checking affected rows

`changeStatus` handlers (`dental.ts:202-228`, `constellation.ts:739-766`) treat a knex update as successful even when zero rows match, and skip audit logging when a single (non-array) ID is passed — so silent no-op status changes are reported as success.

---

8. Dependency vulnerabilities (`npm audit` equivalent)

> Note: `node_modules` are not installed in the working copy, so this section is derived from the declared versions in `package.json`. Run `npm audit` in `src/api` and `src/web` after `npm install` to confirm exact advisory IDs and counts.

API (`src/api/package.json`)

Package	Version	Issue	Severity
`axios`	0.24.0	SSRF & credential leakage on redirect (CVE-2023-45857); ReDoS (CVE-2021-3749); outdated `follow-redirects` transitive CVEs	**High**
`xlsx` (SheetJS)	0.18.5	Prototype pollution (CVE-2023-30533) and ReDoS (CVE-2024-22363); **no fixed version on the public npm registry** — must move to the vendor CDN build	**High**
`mysql`	2.18.1	Unmaintained; superseded by `mysql2`	**Medium**
`sqlite3`	5.1.2	Old; transitive `node-gyp` / `tar` advisories historically	**Medium**
`express`, `express-session`, `connect-redis`	mixed	Generally current but pull transitive advisories; confirm via audit	Low–Medium
`typescript`	4.0.5	Long out of date (toolchain, not runtime)	**Low**

Web (`src/web/package.json`)

Package	Version	Issue	Severity
`vue`	2.7.14	**Vue 2 reached end-of-life (Dec 2023)** — no further security patches	**High**
`vuetify`	2.6.14	Tied to Vue 2 EOL	**High**
`xlsx`	0.18.5	Same prototype pollution / ReDoS as API	**High**
`vue-template-compiler`	2.6.11	Historic XSS/ReDoS advisories; mismatched with Vue 2.7 runtime	**Medium**
`core-js`	3.24.1	Outdated	**Low**

Recommendation: Run `npm audit` / `npm audit --production` in both projects, upgrade or replace the High items first (`axios`, `xlsx`), and budget a Vue 2 → Vue 3 migration as a distinct workstream.

---

9. What a penetration test would likely flag

Mapped to the OWASP Top 10 (2021), a typical engagement against this application would be expected to report:

- **A01 Broken Access Control** — anonymous access to PHI endpoints (CRIT-01), IDOR (CRIT-03), mass-assignment update (CRIT-04), broken admin guard (HIGH-01). *Highest-impact category here.*
- **A02 Cryptographic/Secrets Failures** — secrets baked into images (CRIT-07), weak/default sample secrets (LOW-03), session cookie & secret hygiene (MED-07).
- **A03 Injection** — PL/SQL injection via file data (CRIT-02); path traversal / file-write (CRIT-05).
- **A04 Insecure Design** — authorization bypass footgun (CRIT-06), no per-record authorization model.
- **A05 Security Misconfiguration** — Helmet disabled (HIGH-09), 50 MB bodies (HIGH-08), ineffective rate limiting (HIGH-03), root containers (HIGH-06), exposed Redis (MED-03).
- **A06 Vulnerable & Outdated Components** — §8 dependencies (HIGH-04), EOL runtimes/images (MED-02).
- **A09 Logging & Monitoring Failures** — PHI in logs (LOW-01), forgeable audit entries (CRIT-04), success-on-no-op auditing (LOW-09).

---

10. Suggested remediation roadmap

Phase 0 — Stop-the-bleeding (before any production exposure)

1. Enforce authentication at the router level and add per-record authorization (CRIT-01, CRIT-03).
2. Fix the PL/SQL injection and file-write/path-traversal endpoints (CRIT-02, CRIT-05).
3. Remove secrets from images; rotate any secrets that were ever committed/imaged (CRIT-07).
4. Fix the `SKIP_PERMISSIONS` boolean parsing and the broken admin guard (CRIT-06, HIGH-01).

Phase 1 — Harden

1. Lock down mass-assignment and input validation (CRIT-04, MED-08, MED-05).
2. Generic error responses; enable Helmet; effective rate limiting; lower body limits (HIGH-02, HIGH-09, HIGH-03, HIGH-08).
3. Upgrade/replace vulnerable dependencies (HIGH-04, §8).

Phase 2 — Stabilize & maintain

1. Fix async/response defects and the shared-DB race (HIGH-05, MED-04).
2. Non-root containers, supported base images, Redis auth, CI hardening (HIGH-06, HIGH-07, MED-02, MED-03, MED-09, MED-10).
3. Establish automated test coverage and structured logging (MED-01, LOW-01).
4. Plan the Vue 2 → Vue 3 migration.

11. What the previous team did correctly (for balance)

- **Parameterized queries in the data-access layer** (`UserPermissionRepository`, most route queries) use knex bindings correctly — the injection issues are isolated to the file-data concatenation pattern, not pervasive.
- **The real `.env` is properly gitignored** (`.gitignore:72-75`); only `.env.sample` placeholders are committed.
- **A Content-Security-Policy is configured** (even though full Helmet is disabled).
- **A permissions/role abstraction exists** (`checkPermissions`, `authorize`) — it is simply not applied consistently.
- **CodeQL scanning is wired into CI** (albeit on deprecated action versions).

These indicate the foundation is salvageable; the dominant risk is **inconsistent application of controls that already exist**, plus a small number of genuinely dangerous patterns.

End of report. Findings are based on static review of the repository at the stated date. A dynamic penetration test and a post-`npm install` `npm audit` are recommended to confirm exploitability and exact advisory counts.